

Experiment 7: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Theory:

1. Dataset Source

Dataset used: **Churn Modelling Dataset**

Source: Kaggle

Link: <https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling>

2. Dataset Description

The dataset contains information about bank customers and is used to predict whether a customer will leave the bank (**churn**) or not.

Features:

- CreditScore – Customer credit score
- Geography – Country of customer
- Gender – Male/Female
- Age – Age of customer
- Tenure – Number of years with bank
- Balance – Account balance
- NumOfProducts – Number of products used
- HasCrCard – Credit card availability
- IsActiveMember – Activity status
- EstimatedSalary – Salary of customer

Target Variable:

- Exited (0 = No churn, 1 = Churn)

Dataset Size:

- ~10,000 records
- Mixed data types (numerical + categorical)

3. Mathematical Formulation of ANN

An Artificial Neural Network computes output using:

Neuron Output:

$$Z = \sum (w_i x_i) + b$$
$$A = f(Z)$$

Where:

- w_i = weights
- x_i = input features
- b = bias
- f = activation function

Activation Functions:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$

Loss Function (Binary Crossentropy):

$$Loss = -\frac{1}{N} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

4. Algorithm Limitations

- Requires **large dataset** for better performance
- Computationally expensive
- Difficult to interpret (**black-box model**)
- Sensitive to **hyperparameters**
- May suffer from **overfitting** if not properly regularized

5. Methodology / Workflow

Steps Involved:

1. **Data Collection**
Load dataset from Kaggle
2. **Data Preprocessing**
 - Handle categorical variables (Encoding)
 - Feature scaling using StandardScaler
3. **Train-Test Split**
 - Split data into training and testing sets
4. **Model Building**
 - Create ANN using Sequential model
 - Add input, hidden, and output layers
5. **Model Training**
 - Train model using epochs and batch size
6. **Model Evaluation**
 - Evaluate accuracy and loss
7. **Visualization**
 - Plot accuracy and loss graphs

6. Performance Analysis

- Model achieved accuracy of approximately **86–87%**
- **Accuracy graph** shows increasing trend
- **Loss graph** shows decreasing trend

Interpretation:

- Model is learning effectively
- Training and validation curves are close → **no overfitting**
- Good generalization on unseen data

7. Hyperparameter Tuning

Hyperparameters used:

- **Epochs:** 50
- **Batch Size:** 32
- **Hidden Layers:** 2
- **Neurons:** 6 per layer
- **Optimizer:** Adam

Tuning Impact:

- Increasing epochs improved accuracy
- Proper batch size ensured stable training
- Adam optimizer improved convergence speed

Code:

```
# Step 1: Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Step 2: Load Dataset
df = pd.read_csv("Churn_Modelling.csv")

# Step 3: Data Preprocessing

# Encode Gender
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# One-hot encoding Geography
df = pd.get_dummies(df, columns=['Geography'], drop_first=True)

# Step 4: Select Features
X = df.drop(['RowNumber', 'CustomerId', 'Surname', 'Exited'], axis=1)
y = df['Exited']

# Step 5: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 6: Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Step 7: Build ANN Model
model = Sequential()
```

```
model.add(Dense(units=6, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(units=6, activation='relu'))
model.add(Dense(units=1, activation='sigmoid'))

# Step 8: Compile Model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Step 9: Train Model (with history for graphs)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_test, y_test)
)

# Step 10: Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)

# Step 11: Prediction
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)

# Step 12: Plot Accuracy Graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Training Accuracy', 'Validation Accuracy'])

plt.show()

# Step 13: Plot Loss Graph
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Training Loss', 'Validation Loss'])
```

```
plt.show()
```

Output:

```
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_test, y_test))

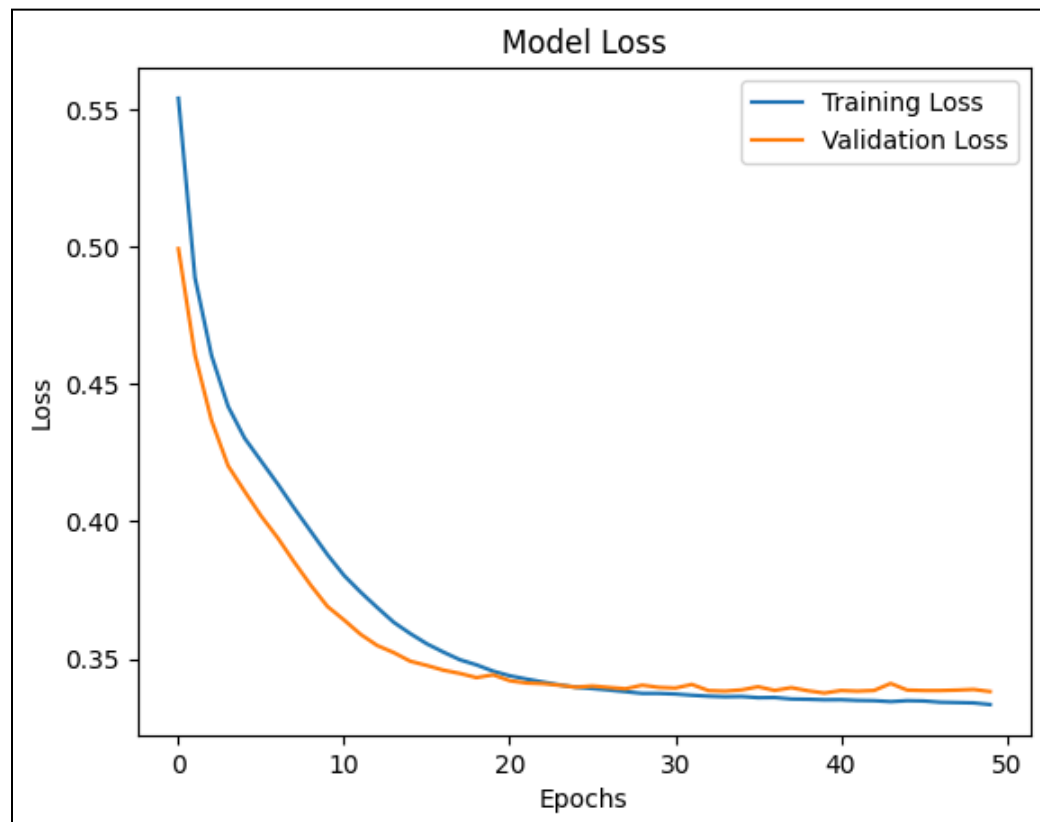
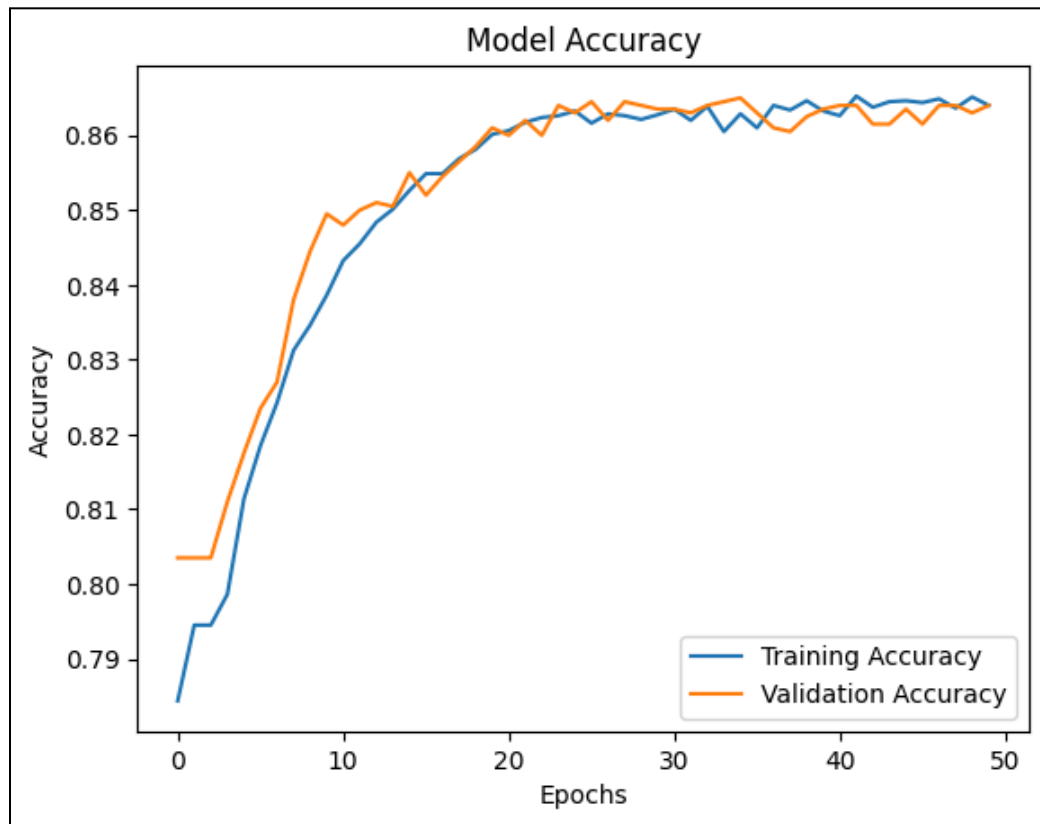
Epoch 1/50
250/250 ————— 4s 6ms/step - accuracy: 0.7844 - loss: 0.5539 - val_accuracy: 0.8035 - val_loss: 0.4992
Epoch 2/50
250/250 ————— 1s 3ms/step - accuracy: 0.7945 - loss: 0.4887 - val_accuracy: 0.8035 - val_loss: 0.4606
Epoch 3/50
250/250 ————— 1s 3ms/step - accuracy: 0.7945 - loss: 0.4604 - val_accuracy: 0.8035 - val_loss: 0.4367
Epoch 4/50
250/250 ————— 1s 3ms/step - accuracy: 0.7986 - loss: 0.4419 - val_accuracy: 0.8110 - val_loss: 0.4203
Epoch 5/50
250/250 ————— 1s 4ms/step - accuracy: 0.8114 - loss: 0.4303 - val_accuracy: 0.8175 - val_loss: 0.4109
Epoch 6/50
250/250 ————— 1s 4ms/step - accuracy: 0.8185 - loss: 0.4219 - val_accuracy: 0.8235 - val_loss: 0.4019
Epoch 7/50
250/250 ————— 1s 5ms/step - accuracy: 0.8242 - loss: 0.4136 - val_accuracy: 0.8270 - val_loss: 0.3940
Epoch 8/50
250/250 ————— 1s 3ms/step - accuracy: 0.8313 - loss: 0.4048 - val_accuracy: 0.8380 - val_loss: 0.3851
Epoch 9/50
250/250 ————— 1s 3ms/step - accuracy: 0.8346 - loss: 0.3963 - val_accuracy: 0.8445 - val_loss: 0.3767
Epoch 10/50
250/250 ————— 1s 3ms/step - accuracy: 0.8386 - loss: 0.3879 - val_accuracy: 0.8495 - val_loss: 0.3691
Epoch 11/50
250/250 ————— 1s 3ms/step - accuracy: 0.8432 - loss: 0.3804 - val_accuracy: 0.8480 - val_loss: 0.3642
Epoch 12/50
250/250 ————— 1s 3ms/step - accuracy: 0.8455 - loss: 0.3743 - val_accuracy: 0.8500 - val_loss: 0.3589
Epoch 13/50
250/250 ————— 1s 3ms/step - accuracy: 0.8484 - loss: 0.3687 - val_accuracy: 0.8510 - val_loss: 0.3549
Epoch 14/50
250/250 ————— 1s 3ms/step - accuracy: 0.8501 - loss: 0.3633 - val_accuracy: 0.8505 - val_loss: 0.3523
Epoch 15/50
250/250 ————— 1s 3ms/step - accuracy: 0.8526 - loss: 0.3592 - val_accuracy: 0.8550 - val_loss: 0.3491
Epoch 16/50
250/250 ————— 1s 3ms/step - accuracy: 0.8549 - loss: 0.3555 - val_accuracy: 0.8520 - val_loss: 0.3476
Epoch 17/50
250/250 ————— 1s 3ms/step - accuracy: 0.8549 - loss: 0.3525 - val_accuracy: 0.8545 - val_loss: 0.3459
Epoch 18/50
250/250 ————— 1s 3ms/step - accuracy: 0.8569 - loss: 0.3497 - val_accuracy: 0.8565 - val_loss: 0.3447
Epoch 19/50
250/250 ————— 1s 3ms/step - accuracy: 0.8581 - loss: 0.3478 - val_accuracy: 0.8585 - val_loss: 0.3432
Epoch 20/50
250/250 ————— 1s 4ms/step - accuracy: 0.8601 - loss: 0.3455 - val_accuracy: 0.8610 - val_loss: 0.3442
Epoch 21/50
250/250 ————— 1s 4ms/step - accuracy: 0.8606 - loss: 0.3439 - val_accuracy: 0.8600 - val_loss: 0.3421
Epoch 22/50
```

```
[10] # Step 10: Evaluate Model
✓ 0s loss, accuracy = model.evaluate(X_test, y_test)
    print("Accuracy:", accuracy)

... 63/63 ————— 0s 3ms/step - accuracy: 0.8640 - loss: 0.3381
    Accuracy: 0.8640000224113464

[11] # Step 11: Prediction
✓ 0s y_pred = model.predict(X_test)
    y_pred = (y_pred > 0.5)

... 63/63 ————— 0s 3ms/step
```



Graph Observation:

Accuracy Graph

- **Training Accuracy** increases from ~78% to ~86–87%
- **Validation Accuracy** follows similar trend
- Both curves are **close together**

Indicates:

- **Good Learning**
- **No Overfitting**
- **High Accuracy**

Loss Graph

- **Training Loss** decreases from ~0.55 to ~0.33
- **Validation Loss** also decreases similarly
- Both curves are **very close**

Indicates:

- **Error Reduction**
- **Stable Model**
- **Proper Training**

Conclusion:

- The **Artificial Neural Network (ANN)** was successfully implemented using **Keras and TensorFlow** for customer churn prediction.
- The model achieved an accuracy of approximately **86–87%**, indicating **good performance**. The graphs showed **increasing accuracy** and **decreasing loss**, which confirms effective learning.
- Both **training and validation curves** were close, indicating **no overfitting** and **good generalization**.
- Thus, ANN proved to be an **efficient and reliable model** for **classification tasks** and can be applied to real-world problems like **customer churn prediction, fraud detection, and recommendation systems**.